CHAPTER 13

Testing Ruby Code

Automated testing has long been an important part of how Ruby developers validate their code. Not only does testing ensure that the code behaves as expected, but also the process of writing tests can expose weaknesses in the structure of the code. Ruby provides a core library called *minitest* to make it easy to write automated tests. However, a more complex and fully-featured library, *RSpec*, is also in extremely common use. The two tools have different terminology and slightly different focus. In this chapter, we'll look at how these tools are used for *unit testing*, which is testing that focuses on small chunks of code, typically individual methods or branches within methods.

Why Unit Test?

It's important to be able to test individual units for many reasons, one of which is that being able to isolate code into testable units is useful for ongoing change and maintenance. Code in one unit often relies on the correct operation of the code in other units. If one unit turns out to contain bugs, then all the code that depends on that unit are potentially affected. This is a big problem.

When you unit test this code as you write it, two things can happen. First, you are more likely to find the bug while the code was still fresh in your mind. Second, because the unit test was only interacting with the code you just wrote, when a bug does appear, you only have to look through a handful of lines of code to find it, rather than doing archaeology on the rest of the code base.

Unit testing helps developers write better code. It helps before the code is actually written, because thinking about testing leads you naturally to create better, more decoupled designs. It helps as you're writing the code, because it gives you instant feedback on how accurate your code is. And it helps after you've written code, both because it gives you the ability to check that the code still works and because it helps others understand how to use your code.

Unit testing is a Good Thing.

Unit testing and dynamic languages such as Ruby go hand in hand. The flexibility of Ruby makes writing tests easy, and the tests make it easier to verify that your code is working. Once you get into the swing of it, you'll find yourself writing a little code, writing a test or two, verifying that everything is copacetic, and then writing some more code. You may even find yourself writing the test before you write a little code.